

Assignment 3: End-to-End ML Versioning with Git, DVC & Google Drive

Fashion-MNIST ANN · Parts A, C, D, E

Munam

github.com/Munam32/MLOps

DVC remote: gdrive_storage ·

tags v1, v2

Part A · Git fundamentals and advanced commands

TASK	WHAT I DID	WHAT IT SHOWS
A1-A2	First commit on <code>main</code> (README + <code>.gitignore</code> for <code>.venv/</code> , <code>__pycache__/</code> , <code>.dvc/cache/</code> , <code>*.h5</code>). All feature work on <code>dev</code> , one commit per script/config.	8 incremental commits on <code>dev</code> before the scenarios.
A3	<code>--oneline --graph --all · --stat -3 · -p -1 · main..dev</code>	All branches as a graph · files and line counts per commit · the actual +/- lines · only commits on <code>dev</code> that <code>main</code> lacks.
A4	<code>git diff</code> , <code>git diff --staged</code> , <code>main..dev</code> , <code>main...dev</code>	Two dots compare the branch tips, so <code>main</code> 's hotfix shows up reversed (-Assignment 3). Three dots compare from where <code>dev</code> branched off, so only <code>dev</code> 's own changes appear (what a pull request shows).
A5	Stashed a half-finished <code>preprocess.py</code> edit, visited <code>main</code> , came back, popped it.	<code>git stash list</code> shows the saved entry before the pop.
A6	hotfix branch fixed the README title, merged into <code>main</code> . <code>git rebase main</code> on <code>dev</code> hit a README conflict, resolved by hand.	Before: <code>dev</code> branches off the old <code>main</code> . After: <code>dev</code> is a straight line on top of the hotfix merge.
A7	Two throwaway commits on <code>scratch</code> , undone with <code>--soft</code> and <code>--hard</code> .	<code>--soft</code> undoes the commit but keeps the change staged. <code>--hard</code> undoes the commit and deletes the change.
A8	<code>git mv</code> the plot into <code>reports/</code> . <code>git rm</code> an obsolete <code>scratch</code> script.	Recorded as a rename (R, history kept) and a deletion (D).

A2/A3 · history of dev

```
$ git log --oneline --graph --all
* 85e40dd feat: add evaluate.py, metrics.json, confusion_matrix.png
* bcbf0cc feat: create train.py
* 8e2f0da chore: modified params.yaml opening structure
* 6e3f3a6 feat: add params.yaml and preprocess.py
... 4 more commits
* 33fa36b Initial Commit

$ git log main..dev --oneline | wc -l
8
```

A4 · git diff variants

```
$ git diff # (a) edited, not staged
-## MLOps Assignment 1
+## MLOps Assignment 1: Fashion-MNIST ANN Pipeline
... 22 more lines

$ git add README.md && git diff # now empty
$ git diff --staged --stat # (b) staged
README.md | 24 ++++++

$ git diff main..dev -- README.md # (c) two dots
-## MLOps Assignment 3
+## MLOps Assignment 1: Fashion-MNIST ANN Pipeline
$ git diff main...dev -- README.md # (c) three dots
-## MLOps Assignment 1
+## MLOps Assignment 1: Fashion-MNIST ANN Pipeline
```

A5 · stash

```
$ git status --short
M src/preprocess.py
$ git stash push -m 'wip: pixel range check in preprocess'
Saved working directory and index state On dev: wip: pixel range check
$ git switch main
$ git switch dev
$ git stash list
stash@{0}: On dev: wip: pixel range check in preprocess
$ git stash pop
modified: src/preprocess.py
Dropped refs/stash@{0} (45d54c2f...)
```

A6 · hotfix, rebase, conflict

```
$ git switch -c hotfix
$ git commit -am 'fix: correct assignment number in README'
$ git switch main && git merge --no-ff hotfix && git branch -d hotfix
$ git switch dev && git rebase main
CONFLICT (content): Merge conflict in README.md
<<<<<< HEAD
## MLOps Assignment 3
=====
## MLOps Assignment 1: Fashion-MNIST ANN Pipeline
>>>>>> (dev)
$ git add README.md && git rebase --continue # kept: Assignment 3 + dev's title
Successfully rebased and updated refs/heads/dev.
```

A6 · before rebase

```
$ git log --oneline --graph --all # BEFORE
* dfdcbc3 chore: add scratch script ...
... 10 more dev commits
* 7d60d03 feat: create prepare.py
| * bdd6d6f Merge branch 'hotfix'
|/|
| * 708ec94 fix: correct assignment number
|/
* 33fa36b Initial Commit
```

A6 · after rebase

```
$ git log --oneline --graph --all # AFTER
* a7ab962 chore: add scratch script ...
... 10 more dev commits (new IDs)
* 96c2748 feat: create prepare.py
* bdd6d6f Merge branch 'hotfix'
| \
| * 708ec94 fix: correct assignment number
|/
* 33fa36b Initial Commit
```

A7 · reset --soft vs --hard

```
$ git reset --soft HEAD~1
$ git status --short
A throwaway1.txt # still staged

$ git reset --hard HEAD~1
HEAD is now at b5e4951 chore: throwaway commit 1
$ git status --short # clean
$ ls throwaway2.txt
ls: cannot access 'throwaway2.txt': No such file
```

A8 · git mv and git rm

```
$ git mv confusion_matrix.png reports/confusion_matrix.png
$ git status --short
R confusion_matrix.png -> reports/confusion_matrix.png

$ git rm scratch_inspect.py
rm 'scratch_inspect.py'
$ git status --short
D scratch_inspect.py
```

Part C · DVC with Google Drive as remote

C1-C3. Installed `dvc[gdrive]` (plus a `pyOpenSSL` upgrade to fix a `GEN_EMAIL` crash), ran `dvc init` on `dev`, and registered the Drive folder with `dvc remote add -d gdrive_storage gdrive://<FOLDER_ID>`. **C4.** Google blocked DVC's built-in sign-in app ("This app is blocked"), so I created my own OAuth client in Google Cloud and stored its ID and secret with `dvc remote modify --local`. That writes them to `.dvc/config.local`, which is git-ignored. The sign-in token lives in `~/.cache/pydrive2fs/`, outside the repo. **C5.** `dvc add data/raw data/processed models` created three small pointer files. Those and the `.gitignore` changes were committed, then `dvc push` uploaded the data.

```
C2/C3 · remote registered in .dvc/config
$ dvc remote list
gdrive_storage  gdrive://10YsvNKN6_F3Htff1tR2YluxfCFKZVRo8  (default)

$ cat .dvc/config
[core]
    remote = gdrive_storage
['remote "gdrive_storage"']
    url = gdrive://10YsvNKN6_F3Htff1tR2YluxfCFKZVRo8

C4 · no credentials in Git
$ cat .dvc/.gitignore
/config.local
/tmp
/cache

$ git check-ignore -v .dvc/config.local
.dvc/.gitignore:1:/config.local .dvc/config.local

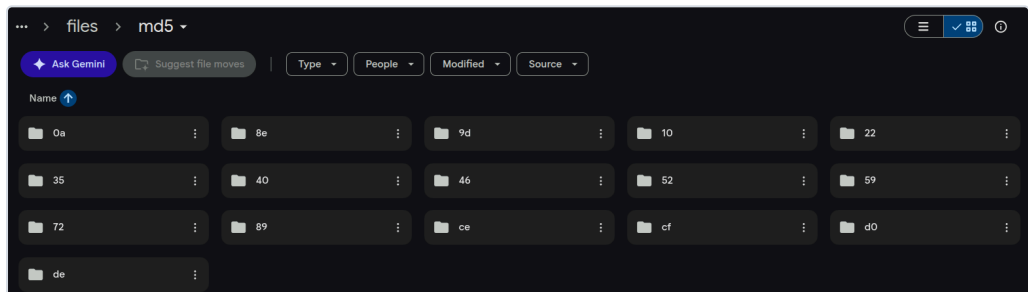
$ git ls-files | grep -i -E 'config.local|token|credential|secret' || echo '(no credential files tracked)'
(no credential files tracked)

$ ls ~/.cache/pydrive2fs/
3391...(client ID folder, holds the sign-in token)
7107...(client ID folder, holds the sign-in token)

C5 · dvc add, pointer file committed
$ git show --stat --format='%h %s' b433fb3
b433fb3 feat: track raw data, processed data, and models

.gitignore      | 3 +--
data/.gitignore | 2 ++
data/processed.dvc | 6 +++++
data/raw.dvc    | 6 +++++
models.dvc      | 6 +++++
5 files changed, 21 insertions(+), 2 deletions(-)

$ git show b433fb3:data/raw.dvc
outs:
- md5: 0a2fc63869fef5432996d7110256447d.dir
  size: 54951020
  nfiles: 2
  hash: md5
  path: raw
```



The Drive remote in the browser. DVC stores files under `files/md5/`, grouped by the first two characters of each file's hash.

Part D · DVC pipeline (`dvc.yaml` + `params.yaml`)

D1-D2. `params.yaml` is the single source of hyperparameters, read with `yaml.safe_load()`. `dvc.yaml` wires four stages by their inputs and outputs. `metrics.json` is declared under `metrics:` so `dvc metrics diff` can compare versions. Both files are shown in their final state (Part E added `normalization`). In Part D, the three `dvc add` pointers from C5 were replaced with `dvc remove`, because the pipeline now owns those folders and records their hashes in `dvc.lock`.

```
Final params.yaml
$ cat params.yaml
preprocess:
  val_size: 0.1
  seed: 42
  normalization: minus_one_one # or zscore

train:
  dense_units: 256
  dropout_rate: 0.2
  learning_rate: 0.001
  epochs: 20
  batch_size: 32

Final dvc.yaml
$ cat dvc.yaml
stages:
  prepare:
    cmd: python src/prepare.py
    deps:
      - src/prepare.py
    outs:
      - data/raw

  preprocess:
    cmd: python src/preprocess.py
    deps:
      - src/preprocess.py
      - data/raw
    params:
      - preprocess.val_size
      - preprocess.seed
      - preprocess.normalization
    outs:
      - data/processed

  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - data/processed
    params:
      - train.dense_units
      - train.dropout_rate
      - train.learning_rate
      - train.epochs
      - train.batch_size
    outs:
      - models

  evaluate:
```

```
cmd: python src/evaluate.py
deps:
- src/evaluate.py
- models
- data/processed
metrics:
- metrics.json:
  cache: false
plots:
- reports/confusion_matrix.png:
  cache: false
```

D3 · dvc repro on a clean checkout (all 4 stages run)

```
$ dvc repro
Running stage 'prepare':
> python src/prepare.py
Saved raw train (60000 samples) and test (10000 samples) data to <clean checkout>/data/raw
Running stage 'preprocess':
> python src/preprocess.py
Saved processed train (54000), val (6000), and test (10000) data to <clean checkout>/data/processed
Running stage 'train':
> python src/train.py
... 20 epochs, final val accuracy 0.8973
Saved trained model and history to <clean checkout>/models
Updating lock file 'dvc.lock'
Running stage 'evaluate':
> python src/evaluate.py
Test loss: 0.3484, test accuracy: 0.8874
Saved metrics.json to <clean checkout> and confusion_matrix.png to <clean checkout>/reports
Updating lock file 'dvc.lock'
$ ls dvc.lock
$ ls dvc.lock
dvc.lock
```

D4 · after dense_units 128 → 256

```
$ dvc status
train:
  changed deps:
    params.yaml:
      modified:      train.dense_units

$ dvc repro
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Running stage 'train':
> python src/train.py
... 20 epochs, final val accuracy 0.9033
Saved trained model and history to ~/Desktop/ML0ps/models
Updating lock file 'dvc.lock'
Running stage 'evaluate':
> python src/evaluate.py
Test loss: 0.3420, test accuracy: 0.8877
Saved metrics.json to ~/Desktop/ML0ps and confusion_matrix.png to ~/Desktop/ML0ps/reports
Updating lock file 'dvc.lock'
```

D4 STAGE	RESULT	WHY
prepare	Skipped	Script and outputs unchanged; it uses no params.
preprocess	Skipped	Depends only on preprocess.* params, which didn't change.
train	Re-ran	Depends on train.dense_units, which changed.
evaluate	Re-ran	Its input models was just rebuilt by train.

DVC compares each stage's current deps and params with the hashes in `dvc.lock`. A stage re-runs only if something it depends on changed, directly or through an earlier stage's new output.

D5 · tag v2, compare, push

```
$ git tag v2
$ git log --oneline --decorate -3
c63beae (HEAD -> dev, tag: v2) exp: increase dense_units from 128 to 256
7c4fcac (tag: v1, origin/dev) feat: add DVC pipeline and replace dvc pointers

$ dvc params diff v1 v2
Path      Param      v1    v2
params.yaml train.dense_units 128    256

$ dvc metrics diff v1 v2
Path      Metric      v1      v2      Change
metrics.json test_accuracy 0.8882  0.8877  -0.0005
metrics.json test_loss    0.3448  0.34202 -0.00278

$ dvc push
Collecting
Pushing
3 files pushed
```

D5 · v1 vs v2

VERSION	DENSE_UNITS	TEST ACC.	TEST LOSS	PARAMS
v1	128	88.82%	0.3448	101,770
v2	256	88.77%	0.3420	203,530

Doubling the hidden layer doubled the model size without improving accuracy (the difference is run-to-run noise). The 128-unit model is already large enough, so v1 remains the better choice.

Part E · Simulated collaboration and conflict resolution

E1-E2. After merging dev (v1, v2) into main, teammate-sim changed normalization to **[-1, 1]** and main changed it to **z-score**. Each ran `dvc repro` and committed. **E3.** Because `data/processed` is a pipeline output, its hash lives in `dvc.lock`, which plays the role of the pointer file.

E3 · code conflict: preprocess.py

```
<<<<<< HEAD
# Standardize with the training set's mean and std (z-score)
train_images = train_raw["images"].astype("float32")
mean, std = train_images.mean(), train_images.std()
train_images = (train_images - mean) / std
train_labels = train_raw["labels"]
test_images = (test_raw["images"].astype("float32") - mean) / std
test_labels = test_raw["labels"]

assert abs(train_images.mean()) < 1e-3, "training pixels not centered on 0"
```

E3 · git merge teammate-sim

```
$ git merge teammate-sim
CONFLICT (content): Merge conflict in dvc.lock
CONFLICT (content): Merge conflict in metrics.json
CONFLICT (content): Merge conflict in reports/confusion_matrix.png
CONFLICT (content): Merge conflict in src/preprocess.py
$ git status --short
```

E3 · data conflict: dvc.lock

```
outs:
- path: data/processed
  hash: md5
<<<<<< HEAD
```

```

=====
# Scale pixels from [0, 255] to [-1, 1] so inputs are centred on zero
train_images = train_raw["images"].astype("float32") / 127.5 - 1.0
train_labels = train_raw["labels"]
test_images = test_raw["images"].astype("float32") / 127.5 - 1.0
test_labels = test_raw["labels"]

assert -1.0 <= train_images.min() and train_images.max() <= 1.0, "pixels not in [-1, 1]"
>>>>> teammate-sim

```

```

md5: cf42c4dbfcff8dd7a675d8a0b389870d.dir
=====
md5: bc407e0136d17f2132f0f4b3a8fcafc0.dir
>>>>> teammate-sim
size: 219591530
nfiles: 3
train:
cmd: python src/train.py
deps:
- path: data/processed
hash: md5
<<<<<< HEAD
md5: cf42c4dbfcff8dd7a675d8a0b389870d.dir
=====
md5: bc407e0136d17f2132f0f4b3a8fcafc0.dir
>>>>> teammate-sim

```

BRANCH	NORMALIZATION	DATA/PROCESSED	TEST ACC.	TEST LOSS
main	z-score	cf42c4...	88.07%	0.3922
teammate-sim	[-1, 1]	bc407e...	88.32%	0.3669

E4. Code: both methods were kept behind a new `preprocess.normalization` param, also added to `dvc.yaml`. **Data:** the teammate version scored better, so it is authoritative. I took its `dvc.lock` with `git checkout --theirs` and ran `dvc checkout`. Re-running `preprocess` with the merged code produced **the identical hash** `bc407e...`, so `train` and `evaluate` were skipped. **E5.** Clean status, merge commit, a second `dvc repro` that skips everything, and both remotes pushed.

```

E4 · resolved code
if params["normalization"] == "minus_one_one":
    # Scale pixels from [0, 255] to [-1, 1] so inputs are centred on zero
    train_images = train_images / 127.5 - 1.0
    test_images = test_images / 127.5 - 1.0
    assert -1.0 <= train_images.min() and train_images.max() <= 1.0, "pixels not in [-1, 1]"
    elif params["normalization"] == "zscore":
        # Standardize with the training set's mean and std
        mean, std = train_images.mean(), train_images.std()
        train_images = (train_images - mean) / std
        test_images = (test_images - mean) / std
        assert abs(train_images.mean()) < 1e-3, "training pixels not centred on 0"
    else:
        raise ValueError(f"unknown normalization: {params['normalization']}")

```

```

E4 · resolved data
$ git checkout --theirs dvc.lock metrics.json reports/confusion_matrix.png
Updated 3 paths from the index

$ grep -A2 'path: data/processed' dvc.lock | head -3
- path: data/processed
  hash: md5
  md5: bc407e0136d17f2132f0f4b3a8fcafc0.dir

$ dvc checkout

$ dvc repro
Stage 'prepare' didn't change, skipping
Running stage 'preprocess':
> python src/preprocess.py
Saved processed train (54000), val (6000), and test (10000) data to ~/Desktop/MLOps/data/processed
Updating lock file 'dvc.lock'
Stage 'train' didn't change, skipping
Stage 'evaluate' didn't change, skipping

```

```

E5 · clean, committed, reproducible, pushed

$ dvc status
Data and pipelines are up to date.

$ git commit -q -m 'Merge branch teammate-sim: reconcile normalization' ...
b0450f8 Merge branch teammate-sim: reconcile normalization

$ dvc repro
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'evaluate' didn't change, skipping
Data and pipelines are up to date.

$ dvc status
Data and pipelines are up to date.

$ dvc push --all-commits
WARNING: failed to collect '0f2e2296f47902a5f1e560248713db1baa865313', skipping
WARNING: failed to collect 'da2542a5cd4b851d196a47e78c152f31b9e42c1a', skipping
... 19 more of these, for commits made before DVC was set up
Collecting
Pushing
14 files pushed

$ git push origin main dev teammate-sim
To github.com:Munam32/MLOps.git
 5301455..ca989a1 dev -> dev
 bdd6d6f..b0450f8 main -> main
* [new branch] teammate-sim -> teammate-sim

```